제안용 샘플 · 예시 형식 — 실제 코드를 받기 전에 만든 견본이에요. 판정, 발견 사항, 파일 위치, 명령 결과, 숫자는 모두 가상이에요. 버전 지원 날짜처럼 출처를 적은 것만 실제로 확인한 사실이에요.

1주차 인수·검수 보고서

현장 서비스 관리 웹, 이대로 넘겨받아도 될까요

대상 공간향기 B2B 현장 서비스 관리 웹 기술 React 18 · NestJS 10 · Prisma 6 · PostgreSQL 17

범위 화면 46개 · API 전체 · 테이블 73개 · 자체 시험 143건 기간 착수 1주차 월~금, 금요일에 전달

판정 (예시)

보완하면 그대로 넘겨받을 수 있어요

다시 짤 필요는 없어 보여요. 문제는 설정과 일부 파일에 몰려 있어요. 다만 키 관리와 로그인 토큰은 서버에 올리기 전에 꼭 고쳐야 해요.

다시 짜지 않아도 되는 이유

- ✓ DB 설계(테이블 73개)가 9개 업무 흐름과 맞아요
- ✓ 비밀번호 해시와 개인정보 암호화가 이미 들어가 있어요
- ✓ 자체 시험 143건 중 139건이 실제 DB에서도 통과해요

그대로 인수	지금 판정 **보완 후 인수**	일부 다시 짜기	전면 다시 짜기
치명·높음 0건, 설명서대로 새 서버에 바로 설치됨	제안 범위 안에서 고칠 수 있는 문제만 있음	한 기능은 고치기보다 새로 짜는 게 빠름, 따로 협의	DB 설계가 업무 흐름과 맞지 않음

오픈 전에 꼭 고칠 것

6일

8건 · 제안서 ②③ 범위 안이에요

K-01 T-01 T-02 A-01 C-01 S-01 D-01
M-01 Q-01 Q-02 E-01 L-01 D-02 A-02

● 치명 2 ● 높음 5 ● 중간 5 ● 낮음 2

남은 공수 **10**일

● **치명·높음 7건**이 남아 있어 아직 서버에 올리면 안 돼요. 칸을 누르면 해당 항목으로 가요.

이렇게 확인했어요

받은 설명서만 보고 새 서버에 처음부터 설치해 보는 데서 시작해요. 개발 PC에서 되는 것과 서버에서 되는 것은 달라서요.

1주차 5일

월	**처음부터 설치해 보기**	인계서만 보고 새 서버에 설치·빌드하고, 빈 DB에 구조를 만들어요.
화	**자동 검사**	시험 143건 다시 실행, 의존성 취약점, 키 노출, 보안 규칙 검사.
수	**직접 읽기**	로그인·토큰·암호화 코드를 읽고, 5개 팀 권한표와 API를 맞춰 봐요.
목	**데이터 넣어 보기**	가짜 데이터 1만 건으로 화면별 쿼리 수와 응답 시간을 재요.
금	**보고서와 설명**	이 보고서를 드리고 영등포에서 1시간 설명, 2주차 계획을 정해요.

통과했다던 시험, 다시 돌려 봤어요

시험	받은 결과	다시 돌린 결과
백엔드 시나리오	83 통과	79 통과 · 4 실패
API 연결	60 통과	60 통과
화면 시험	통과	패드 1대 확인

백엔드 시험 83건 중 52건이 실제 DB 대신 가짜(mock)를 쓰고 있었어요. 실제 DB로 바꿔 돌리니 4건이 실패했어요. AI가 시험을 통과시키려고 가짜를 넣는 일이 흔해서 꼭 봐요. (예시)

AI가 쓴 NestJS·Prisma 코드에서 꼭 보는 8가지

저도 AI 개발 도구로 일해서, AI가 어디서 자주 빠뜨리는지 알아요. 화면에서 잘 돌아가도 아래 8가지는 따로 확인해요.

영역	무엇을 보나	AI 코드에서 잘 생기는 이유	확인 방법	결과 (예시)
보안 기본 로그인·권한·입력 검사	• 모든 API에 로그인 확인이 걸려 있는지 • 5개 팀 권한표대로 막히는지 • 모르는 칸(예: role)을 몰래 보내도 버리는지 • 로그인 시도 횟수 제한	화면에 버튼이 없는 API는 막는 걸 자주 빠뜨려요. 화면 시험으로는 드러나지 않아요.	팀별 시험 계정으로 API를 하나씩 직접 호출	높음 2
환경변수·키 암호화 키, 비밀번호	• 코드와 저장소 기록(git)에 키가 남았는지 • 키가 없을 때 기본값으로 켜지지 않는지 • 같은 값이 늘 같은 암호문이 되지 않는지 • 키를 바꾸는 방법이 있는지	오류를 물어보며 .env를 통째로 붙여 넣는 일이 많아, 개발용 키가 여기저기 남아요.	저장소 전체 기록 키 검사, 암호화 함수 직접 읽기	치명 1
로그인 토큰 JWT 방식	• 토큰 유효 시간과 저장 위치 • 로그아웃·퇴사 때 바로 막히는지 • 비밀번호를 바꾸면 다른 기기가 끊기는지	예제처럼 '오래 가는 토큰 하나'로 끝내는 경우가 많아요.	토큰을 꺼내 만료·폐기 동작을 직접 시험	치명 1 높음 1
CORS·배포 설정 운영 모드	• 모든 주소 허용 + 쿠키 허용인지 • 개발 서버(vite dev, start:dev)로 돌고 있지 않은지 • API 문서 화면이 운영에 열려 있지 않은지 • 보안 헤더	패드에서 같은 와이파이로 접속 시험을 하려고 '모두 허용'으로 풀어 둔 채 남기 쉬워요.	설정 파일 확인, 다른 주소에서 요청 보내 보기	높음 1
N+1 쿼리 느려지는 목록	• 반복문 안에서 DB를 한 건씩 부르는지 • 목록에 페이지 나누기가 있는지 • 관계 칸 인덱스 • 필요 없는 칸까지 다 가져오는지	데이터가 몇 건뿐인 개발 PC에서는 느린 게 보이지 않아요. 실제 데이터가 들어가야 드러나요.	쿼리 기록을 켜고 화면마다 쿼리 수 세기	중간 2
마이그레이션 DB 구조 변경 기록	• 기록과 실제 DB 구조가 같은지 • 빈 DB에서 한 번에 만들어지는지 • db push·reset으로 맞춰 오지 않았는지 • 시드에 고정 관리자 비밀번호가 없는지	스키마 오류가 나면 DB를 지우고 다시 맞추는 방법을 자주 써요. 운영에서는 데이터가 지워져요.	빈 DB에 migrate deploy, 구조 차이 비교	중간 1
에러 처리·기록 오류 응답, 로그	• DB 오류를 알맞은 문구로 바꾸는지 • 응답에 SQL·내부 이름이 보이지 않는지 • 로그에 전화번호·주소가 찍히지 않는지 • 개인정보 접속기록	파일마다 오류 처리 방식이 다르고, 디버깅하려고 요청 내용을 통째로 찍어 둔 줄이 남아요.	일부러 틀린 값 보내기, 로그 파일 검색	중간 2
의존성 취약점 Node·라이브러리 버전	• 알려진 취약점 (npm audit) • 지원이 끝난 버전 • 없는 패키지나 이름이 비슷한 가짜 패키지 • lock 파일로 같은 버전이 설치되는지	AI는 배운 시점의 버전을 써서 처음부터 오래된 버전일 때가 있어요. 없는 패키지 이름을 지어내기도 해요.	npm audit, 패키지 실제 존재·다운로드 수 확인	높음 1 낮음 1

발견 사항 14건

모두 예시예요. 줄을 누르면 다시 일으키는 방법과 고친 뒤 확인 결과가 펼쳐져요. 실제 보고서에는 파일·행 위치와 확인 화면 캡처도 함께 들어가요.

● 치명	● 높음	● 중간	● 낮음
지금 바로 정보가 새거나, 남이 관리자처럼 쓸 수 있어요.	조건이 맞으면 정보가 새거나 서비스가 멈춰요.	데이터가 쌓이면 느려지거나, 운영·법 기준에 못 미쳐요.	당장 문제는 없지만 나중에 고치는 비용을 올려요.
발견한 날 바로 알리고, 오픈 전 반드시	오픈 전 반드시	오픈 전 권장, 늦어도 오픈 후 1개월	안정화 기간

번호·심각도	내용	고치는 방법	공수	언제
K-01 키 관리 ● 치명	**개인정보 암호화 키가 저장소 기록에 남아 있어요** 지금 파일에서는 지웠지만 예전 커밋에 그대로 있어요. 소스를 받은 사람은 누구나 주소·전화번호를 풀 수 있어요. `.env · 커밋 a3f9c1e`	운영용 키를 새로 만들고 기록에서 지워요. 실제 데이터가 아직 없어서 지금 바꾸면 다시 암호화할 것도 없어요. 키 사본은 대표님이 따로 보관해요.	0.5 일	오픈 전 필수

다시 일으키는 방법

지금 파일에는 없지만, 저장소 전체 기록에서 키 이름을 찾으면 나와요.

```
$ git log --all -p -S "FIELD_ENC_KEY" -- .env
commit a3f9c1e
+FIELD_ENC_KEY=9c41e0…  (64자리, 보고서에서는 가림)
```

고친 뒤 확인

새 키로 바꾼 뒤 저장소 전체 기록을 다시 검사하고, 옛 키로는 시험 데이터가 풀리지 않는지 봐요.

```
$ gitleaks detect --log-opts="--all"
INF no leaks found
```

번호·심각도	내용	고치는 방법	공수	언제
T-01 로그인 토큰 ● 치명	**토큰 서명 키에 기본값 'secret'이 들어 있어요** 서버 설정에서 키가 빠지면 'secret'으로 켜져요. 이 값을 아는 사람은 관리자 토큰을 직접 만들 수 있어요. `src/auth/jwt.strategy.ts:14`	키가 없으면 서버가 아예 켜지지 않게 바꾸고, 길고 무작위인 키를 써요.	0.5 일	오픈 전 필수

문제 줄과 고친 줄

src/auth/jwt.strategy.ts

```
13      ignoreExpiration: false,
14 -    secretOrKey: process.env.JWT_SECRET || 'secret',
14 +    secretOrKey: config.getOrThrow<string>('JWT_SECRET'),
```

다시 일으키는 방법

시험 서버에서 JWT_SECRET을 빼고 켠 뒤, 'secret'으로 직접 서명한 관리자 토큰으로 직원 목록을 불러요.

```
# $FAKE = 'secret'으로 서명한 관리자 토큰
$ curl -H "Authorization: Bearer $FAKE"
/api/users
HTTP/1.1 200 OK · 직원 18명
```

고친 뒤 확인

키 없이 켜면 서버가 시작되지 않아요(의도한 동작). 같은 가짜 토큰은 401이에요.

```
# JWT_SECRET 없이 켜기
$ npm run start:prod
TypeError: Configuration key "JWT_SECRET" does
not exist
```

T-02
로그인 토큰
높음

로그인 토큰이 7일 동안 살아 있고, 로그아웃해도 막히지 않아요

토큰을 브라우저 저장소에 둬요. 패드를 잃어버리거나 직원이 퇴사해도 7일 동안 접속돼요.

`src/auth/auth.service.ts`

15분짜리 접속 토큰과 쿠키(httpOnly)에 담은 갱신 토큰으로 바꿔요. 요청마다 계정 상태를 확인해서 계정을 끄면 바로 막혀요.

2 일 · 오픈 전 필수

다시 일으키는 방법

로그인해서 브라우저 저장소의 토큰을 복사하고, 로그아웃한 뒤 그 토큰으로 API를 불러요. 관리자 화면에서 계정을 꺼도 같아요.

```
# 로그아웃 뒤, 복사해 둔 토큰으로
$ curl -H "Authorization: Bearer $OLD"
/api/visit-logs
HTTP/1.1 200 OK   (만료까지 7일)
```

고친 뒤 확인

로그아웃·계정 끄기 직후 같은 토큰은 바로 막히고, 브라우저 저장소에는 토큰이 남지 않아요.

```
$ curl -H "Authorization: Bearer $OLD"
/api/visit-logs
HTTP/1.1 401 Unauthorized
```

A-01
권한
높음

직원 목록 API가 모든 직원의 전화·이메일을 돌려줘요

로그인만 하면 팀과 관계없이 18명 전체 연락처가 내려와요. 화면에는 이름만 보여서 시험에서 드러나지 않았어요.

`GET /api/users`

팀별 권한을 확인하고, 화면에 필요한 칸만 내보내요. 같은 방식으로 API 전체를 권한표와 맞춰 봐요.

0.5 일 · 오픈 전 필수

다시 일으키는 방법

연락처 권한이 없는 조향팀 시험 계정으로 직원 목록을 불러요.

```
$ curl -H "Authorization: Bearer $SCENT_TEAM"
/api/users
[{ "name": "김○○", "team": "SCM", "phone": "010-
****-1234", "email": "k***@…" }, … 18명]
```

고친 뒤 확인

5개 팀 계정으로 모든 API를 불러 본 결과를 권한표로 남겨요. 직원 목록에는 이름과 팀만 내려와요.

```
[{ "id": 7, "name": "김○○", "team": "SCM" }, …]
```

C-01
CORS
높음

모든 사이트의 요청을 쿠키까지 받아 주도록 열려 있어요

패드 접속 시험 때 '모든 주소 허용'으로 풀어 둔 설정이 그대로 있어요. T-02에서 토큰을 쿠키로 옮기면 다른 사이트가 직원의 로그인 상태로 API를 부를 수 있게 돼서, T-02보다 먼저 닫아요.

`src/main.ts` `enableCors({ origin: true, credentials: true })`

화면과 API를 같은 주소로 내보내 CORS가 필요 없게 해요. 꼭 필요하면 회사 주소 하나만 허용해요.

0.5 일 · 오픈 전 필수

문제 줄과 고친 줄

src/main.ts

```
8      app.setGlobalPrefix('api');
9  -   app.enableCors({ origin: true, credentials: true });
9  +   // 화면과 API를 같은 주소(/api)로 내보내 CORS를 열지 않아요
```

다시 일으키는 방법

아무 사이트 주소를 넣은 요청을 보내면, 서버가 그 주소를 그대로 허용해 돌려줘요.

```
$ curl -I -H "Origin: https://아무-사이트.example"
/api/users
Access-Control-Allow-Origin: https://아무-사이
트.example
Access-Control-Allow-Credentials: true
```

고친 뒤 확인

같은 요청에 허용 머리글이 붙지 않고, 패드 화면은 같은 주소의 /api로 문제없이 불러요.

```
$ curl -I -H "Origin: https://아무-사이트.example"
/api/users
(Access-Control-Allow-Origin 없음)
```

S-01
보안 기본
높음

로그인 시도 횟수 제한이 없어요

비밀번호를 계속 바꿔 넣어 볼 수 있어요.

`src/auth/auth.controller.ts`

1분에 5번까지만 받고, 계속 틀리면 잠시 잠가요.

0.5 일

오픈 전 필수

다시 일으키는 방법

틀린 비밀번호로 1분 안에 20번 로그인해 봐요.

```
# POST /api/auth/login × 20 (1분 안)
401 × 20   한 번도 막히지 않음
```

고친 뒤 확인

6번째부터 막히고, 잠금이 풀린 뒤에는 맞는 비밀번호로 다시 들어가져요.

```
# POST /api/auth/login × 20 (1분 안)
401 × 5  →  429 Too Many Requests × 15
```

D-01
의존성
높음

Node 20을 쓰고 있어요 (2026년 4월 30일 지원 종료)

더 이상 보안 패치가 나오지 않아요. 종료 날짜는 Node.js 공식 일정에서 확인한 사실이에요.

`package.json · .nvmrc`

Node 24 LTS(2028년 4월까지 지원)로 올리고 자체 시험 143건을 다시 돌려요.

1 일

오픈 전 필수

다시 일으키는 방법

서버와 설정 파일의 Node 버전을 봐요.

```
$ cat .nvmrc
20
```

고친 뒤 확인

Node 24에서 새로 설치·빌드하고, 시험 143건 결과를 올리기 전과 나란히 적어 바뀐 것이 없는지 봐요.

```
$ cat .nvmrc
24
```

M-01
마이그레이션
중간

마이그레이션 기록 없이 db push로 DB를 맞춰 왔어요

서버 DB를 같은 구조로 만들 정확한 순서가 없어요. 나중에 칸을 바꾸면 운영 데이터가 지워질 수 있어요.

`prisma/migrations 폴더 없음`

지금 구조를 기준점으로 기록하고, 서버에는 migrate deploy만 쓰도록 자동 배포에 넣어요.

0.5 일

오픈 전 필수

다시 일으키는 방법

기록된 마이그레이션이 있는지 봐요.

```
$ npx prisma migrate status
No migration found in prisma/migrations
```

고친 뒤 확인

빈 DB에 기록된 순서대로만 만들고, 지금 스키마와 구조 차이가 없는지 비교해요.

```
$ npx prisma migrate deploy
$ npx prisma migrate diff …  # 빈 DB에 만든 결과 ↔ 스키마
No difference detected.
```

Q-01
N+1 쿼리
중간

방문일지 목록 50건에 쿼리가 101번 나가요

반복문 안에서 고객사와 담당자를 한 건씩 다시 불러와요. 지금은 빠르지만 1년치가 쌓이면 느려져요.

`src/visit-log/visit-log.service.ts:62`

관계 데이터를 한 번에 가져오고, 목록에 페이지 나누기를 넣어요.

0.5 일

오픈 전 권장

다시 일으키는 방법

쿼리 기록을 켜고 방문일지 목록을 한 번 열어 세어요.

```
prisma:query SELECT … FROM "VisitLog" … LIMIT 50
prisma:query SELECT … FROM "Customer" WHERE "id" = $1   × 50
prisma:query SELECT … FROM "User" WHERE "id" = $1       × 50
합계 101번
```

고친 뒤 확인

같은 화면을 다시 열어 쿼리 수를 세요. 목록은 50건씩 나눠 불러요.

```
합계 3번
```

Q-02
인덱스
중간

관계 칸 21곳에 인덱스가 없어요

PostgreSQL은 외래키 칸에 인덱스를 자동으로 만들지 않아요. 고객사별 이슈 조회가 표 전체를 훑고 있어서, 지금은 빨라도 데이터가 쌓이는 만큼 같이 느려져요.

`prisma/schema.prisma`

자주 찾는 칸에 인덱스를 더하고, 가짜 데이터를 늘려 가며 다시 재요.

0.5 일

오픈 전 권장

다시 일으키는 방법

고객사별 이슈 조회의 실행 계획을 봐요.

```
=# EXPLAIN ANALYZE SELECT … FROM "Issue" WHERE "customerId" = 42;
Seq Scan on "Issue"   (표 전체를 훑음)
```

고친 뒤 확인

같은 조회가 인덱스를 타는지 보고, 1만 건과 10만 건에서 시간을 다시 재요.

```
Index Scan using "Issue_customerId_idx" on "Issue"
```

E-01
에러 처리
중간

DB 오류 문구가 그대로 화면에 나가고, 칸 이름까지 보여요

catch에서 Prisma 오류 문구를 그대로 응답에 넣고 있어요. 같은 장비 번호를 두 번 넣으면 영어 오류와 DB 칸 이름이 직원 화면에 떠요.

`장비·고객사 서비스 catch 12곳`

Prisma 오류 코드를 안내 문구로 바꾸는 전역 필터를 하나 두고, 내부 이름은 서버 기록에만 남겨요.

0.5 일

오픈 전 권장

문제 줄과 고친 줄

src/equipment/equipment.service.ts

```
41    } catch (e) {
42  -   throw new BadRequestException(e.message);
42  +   throw e; // 전역 필터가 P2002를 409 안내 문구로 바꿔요
```

다시 일으키는 방법

같은 장비 번호로 두 번 등록해요.

```
# POST /api/equipment {"serialNo":"SC-2210-0042"} 두 번째
400 {"message":"… Unique constraint failed on the fields: (`serial_no`)"}
```

고친 뒤 확인

같은 요청에 안내 문구만 나가요. 칸 이름은 서버 기록에만 남아요.

```
409 {"message":"이미 등록된 장비 번호예요"}
```

L-01
접속기록
중간

누가 어떤 고객의 개인정보를 봤는지 기록이 없어요

개인정보 안전성 확보조치 기준 제8조는 접속기록을 1년 이상 보관하라고 해요. 이 시스템에 어떻게 적용되는지는 확인이 필요해요.

고객 연락처 조회와 내려받기를 기록하는 테이블을 두고 1년 보관해요.

1 일

오픈 전 권장

다시 일으키는 방법

고객 상세(연락처) 화면을 연 뒤, DB와 로그에서 누가 언제 봤는지 찾아요.

```
=# SELECT * FROM access_log ORDER BY at DESC LIMIT 1;
ERROR:  relation "access_log" does not exist
```

고친 뒤 확인

연락처를 열거나 내려받을 때마다 직원·시각·고객·접속 주소가 한 줄씩 남아요.

```
11-12 10:41 · 서비스관리 박○○ · 고객사 #118 연락처 조회 · 10.0.3.21
```

D-02
의존성
● 낮음

prisma@latest로 설치하면 8.0 정식 전 버전이 깔려요

10월 21일 npm에서 latest 표시가 8.0.0-rc.24를 가리키는 걸 실제로 확인했어요. 설치할 때마다 버전이 달라질 수 있어요.

`README 설치 안내`

버전을 정확히 고정하고, 서버에서는 lock 파일 그대로 설치해요(npm ci).

0.5 일 오픈 전 권장

다시 일으키는 방법

npm에서 latest 표시가 어느 버전인지 봐요. (10월 21일 실제 결과)

```
$ npm view prisma dist-tags.latest
8.0.0-rc.24
```

고친 뒤 확인

package.json에 정확한 버전이 적혀 있고, 서버에서는 lock 파일 그대로 설치돼요.

```
"prisma": "6.19.3"   # ^ ~ latest 없이
```

A-02
AI 코드 정리
● 낮음

같은 날짜 변환 함수가 7곳에 따로 있어요

조금씩 달라서 한 곳만 고치면 화면마다 날짜가 달라질 수 있어요. 쓰이지 않는 파일도 38개 있어요.

`web/src/utils/ 외 6곳`

하나로 모으고 한국 시간 기준 시험을 더해요. 안 쓰는 파일은 지워요.

1 일 안정화 기간

다시 일으키는 방법

같은 시각을 7개 함수에 넣어 결과를 나란히 봐요.

```
# 2026-11-12 08:30 (한국 시간)을 넣으면
2026-11-12 · 2026. 11. 12. · 11월 12일   (3가지 모양)
```

고친 뒤 확인

함수 하나로 모은 뒤, 한국 시간 자정 앞뒤 시험 4건을 더하고 빌드·시험 143건을 다시 돌려요.

```
날짜 함수 1곳 · 시험 4건 통과
```

고치는 순서와 공수

반나절 단위로 적어요. 제안서의 ② 서버 구축(6일)과 ③ 보안 보완(8일) 안에 들어가고, 넘치면 먼저 여쭤보고 정해요.

언제 고치나

오픈 전 필수 · 8건 — **6 일**

치명·높음 전부와 마이그레이션 기준점

오픈 전 권장 · 5건 — **3 일**

느려짐 대비, 오류 문구, 접속기록, 버전 고정

안정화 기간 · 1건 — **1 일**

중복 코드 정리 (오픈 후 3개월 안에)

합계 14건 · 10일 (예시)

오픈 전 필수 8건은 이 순서로 고쳐요

1. 키부터 바꿔요
   K-01, **T-01** · 다른 수정이 새 키 위에서 이뤄지게
2. 문을 닫아요
   A-01, **C-01**, **S-01** · 권한, CORS, 로그인 시도 제한. 토큰을 쿠키로 옮기기 전에 닫아야 해서 3번보다 먼저
3. 토큰 방식을 바꿔요
   T-02 · 가장 큰 작업이라 따로 시험 기간을 둬요
4. 버전을 올려요
   D-01 · Node 24 LTS, 올린 뒤 시험 143건 다시 실행
5. 배포 순서를 고정해요
   M-01 · 서버에는 기록된 마이그레이션만 적용

결정해 주실 것

2주차 서버 구축 전에 정하면 좋은 것들이에요.

회사 메일이 Google Workspace인가요?

구글 캘린더 연동 앱을 '테스트' 상태로 두면 연결이 7일마다 끊겨요(구글 공식 문서). Workspace라면 '내부 앱'으로 설정해 구글 검수 없이 쓸 수 있어요.

현장 패드를 여러 직원이 같이 쓰나요?

같이 쓰면 자동 로그아웃 시간을 짧게 정해야 해요.

메일은 어느 계정으로 보낼까요?

회사 메일 계정으로 보낼지, 발송 서비스를 쓸지 정해요. 스팸함에 안 들어가게 도메인 설정(SPF·DKIM)도 함께 해요.

도메인

회사 도메인이 있으면 하위 주소(예: field.회사도메인)로 추가 비용 없이 써요.

기존 자료

파일 형식과 대략의 건수를 알려 주시면 이관 리허설을 미리 해 봐요.

실제 보고서는 1주차 금요일에 이 형식으로 드려요

판정과 발견 사항은 받은 코드로 새로 채우고, 영등포에서 1시간 동안 같이 보며 설명해요.